Server-side scripting

Introduction

1. Web servers are used to execute server-side scripting.
2. They are basically used to create dynamic pages.
3. It can also access the file system residing at the webserver.
4. Scripts can be written in any of a number of server-side scripting languages available.
5. It is used to retrieve and generate content for dynamic pages.
6. When you need to store and retrieve information a database will be used to contain data. It can use huge resources of the server.
7. It reduces client-side computation overhead.
8. The server sends pages to the request of the user/client.

Server-side Programming

It is the program that runs on server dealing with the generation of content of web page.

1) Querying the database
2) Operations over databases
3) Access/Write a file on server.
4) Interact with other servers.
5) Structure web applications.
6) Process user input.

Client and server side scripting

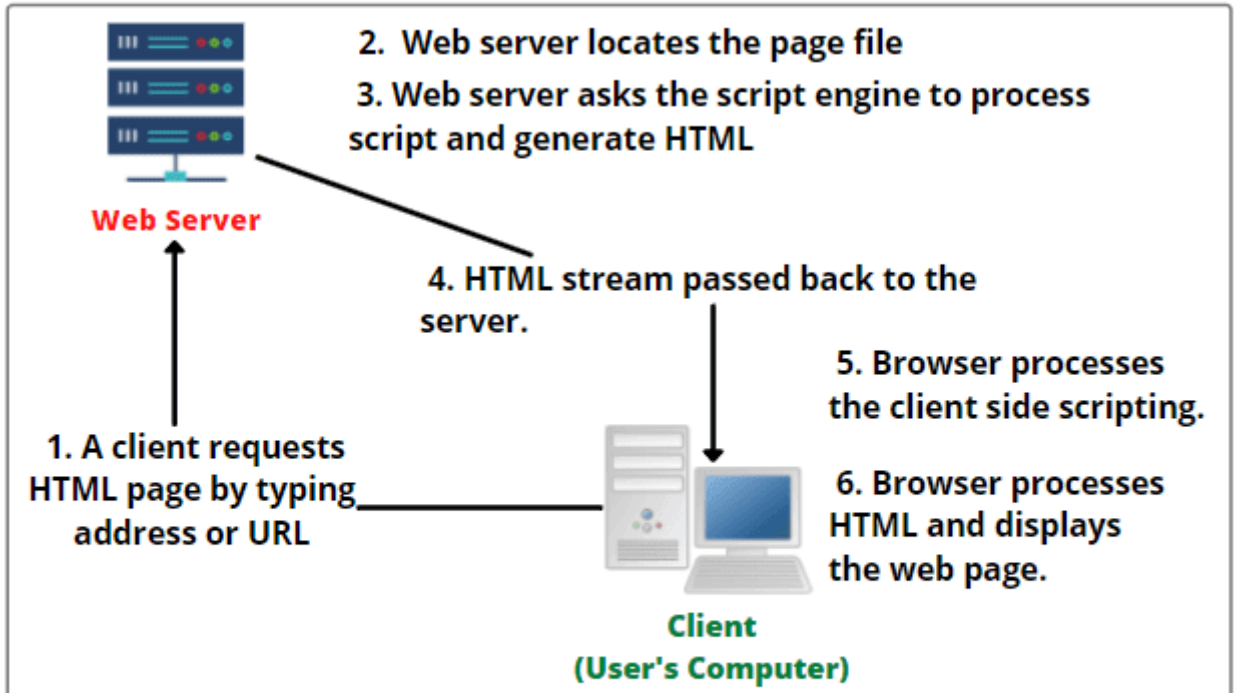


Figure: Server-side and Client-side scripting

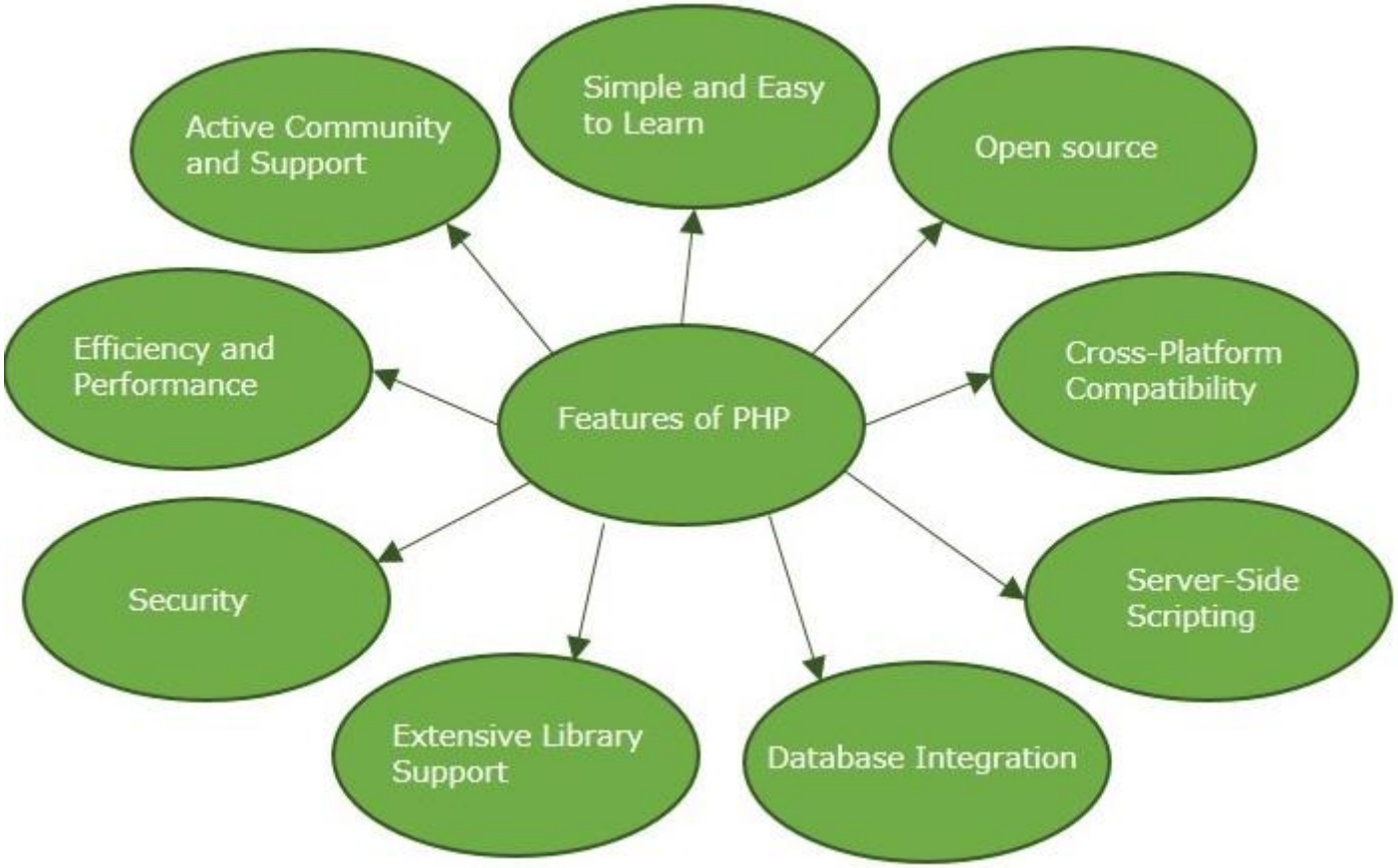
PHP: Hypertext Preprocessor

- HyperText means text containing all sorts of web markups, and PreProcessor means all of the HyperText is processed first.
- Then, the result is sent as pure HTML to the web browser. A client cannot see the PHP source code because it is preprocessed and interpreted.
- PHP (Hypertext Preprocessor) is an open-source HTML-embedded server-side scripting language used to develop dynamic and interactive web applications and as a general-purpose programming language.

PHP: Hypertext Preprocessor

- PHP has many syntaxes similar to C, Java, and Perl and has many unique features and specific functions.
- PHP page is a file with a .php extension can contain a combination of HTML Tags and PHP scripts.
- PHP is Server-side scripting language: Server-side scripting means that the PHP code is processed on the web server rather than the client machine.
- PHP supports many databases.

Features of PHP



Features of PHP

- PHP performs system functions. It can create, open, read, write, and close the files.
- PHP can handle forms. It can gather data from files, save data to a file, through email you can send data, return data to the user.
- You add, delete, modify elements within your database through PHP.
- Access cookies variables and set cookies.
- Using PHP, you can restrict users to access some pages of your website.
- It can encrypt data.

PHP - Syntax

- A ".php" file may contain HTML, CSS and JavaScript code blocks along with the PHP code. Hence, the PHP parser must differentiate between the PHP code from the other elements.

Example Code

phpfirst.html

phpfirst.html > html

```html
<!DOCTYPE html>
<html>
<body>
    <h1>My PHP Website</h1>
    <?php
        echo "Hello World!";
    ?>
</body>
</html>
```

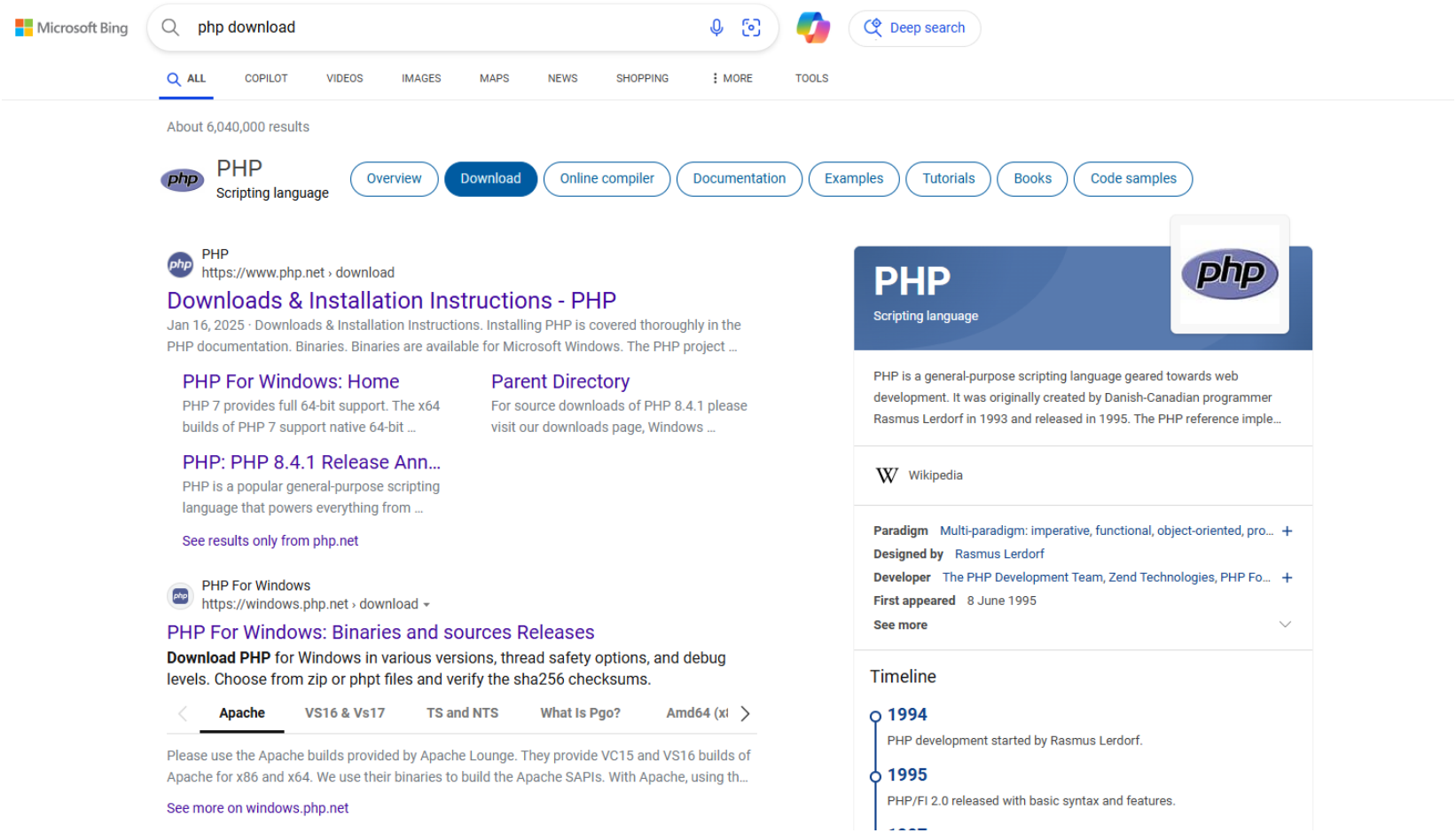
Syntax for comments in PHP code:

```
// This is a single-line comment

# This is also a single-line comment

/* This is a
multi-line comment */
```

Search on browser for php download



Select the link

https://windows.php.net/download/

PHP: Hypertext Preprocessor

Home | Downloads | QA Releases | Snapshots | Team | PHP.net site

PHP For Windows

This site is dedicated to supporting PHP on Microsoft Windows. It also supports ports of PHP extensions or features as well as providing special builds for the various Windows architectures.

If you like to build your own PHP binaries, instructions can be found on the Wiki.

PECL For Windows

PECL extensions for Windows is being worked on. Windows DLL can be downloaded right from the PECL website.

Which version do I choose?

IIS

If you are using PHP as FastCGI with IIS you should use the Non-Thread Safe (NTS) versions of PHP.

Apache

Please use the Apache builds provided by Apache Lounge. They provide VC15 and VS16 builds of Apache for x86 and x64. We use their binaries to build the Apache SAPIs.

With Apache, using the apache2handler SAPI, you have to use the Thread Safe (TS) versions of PHP.

VS16 & VS17

More recent versions of PHP are built with VS16 or VS17 (Visual Studio 2019 or 2022 compiler respectively) and include improvements in performance and stability.

- The VS16 and VS17 builds require to have the *Visual C++ Redistributable for Visual Studio 2015-2022* x64 or x86 installed

Binaries and sources Releases

Select an option to direct access...

Past releases

PHP 8.4 (8.4.3)

Download source code [29.17MB]

Download tests package (phpt) [17.22MB]

VS17 x64 Non Thread Safe (2025-Jan-15 11:07:36)

- Zip [32.18MB]
  sha256: 56e3b4015fe5a429625e5dd96645ccab2c4f42083ba30a4161a11484dd750972
- Debug Pack [37.01MB]
  sha256: 771223dc963dca0c00dc4d9452a2b1f4b727dd69837b3ac99a803aaf138b7992
- Development package (SDK to develop PHP extensions) [1.35MB]
  sha256: 5800f6c106489b45ab3b9d8b6f44ba7f1d54683075d86112c580a6bc8d134caa

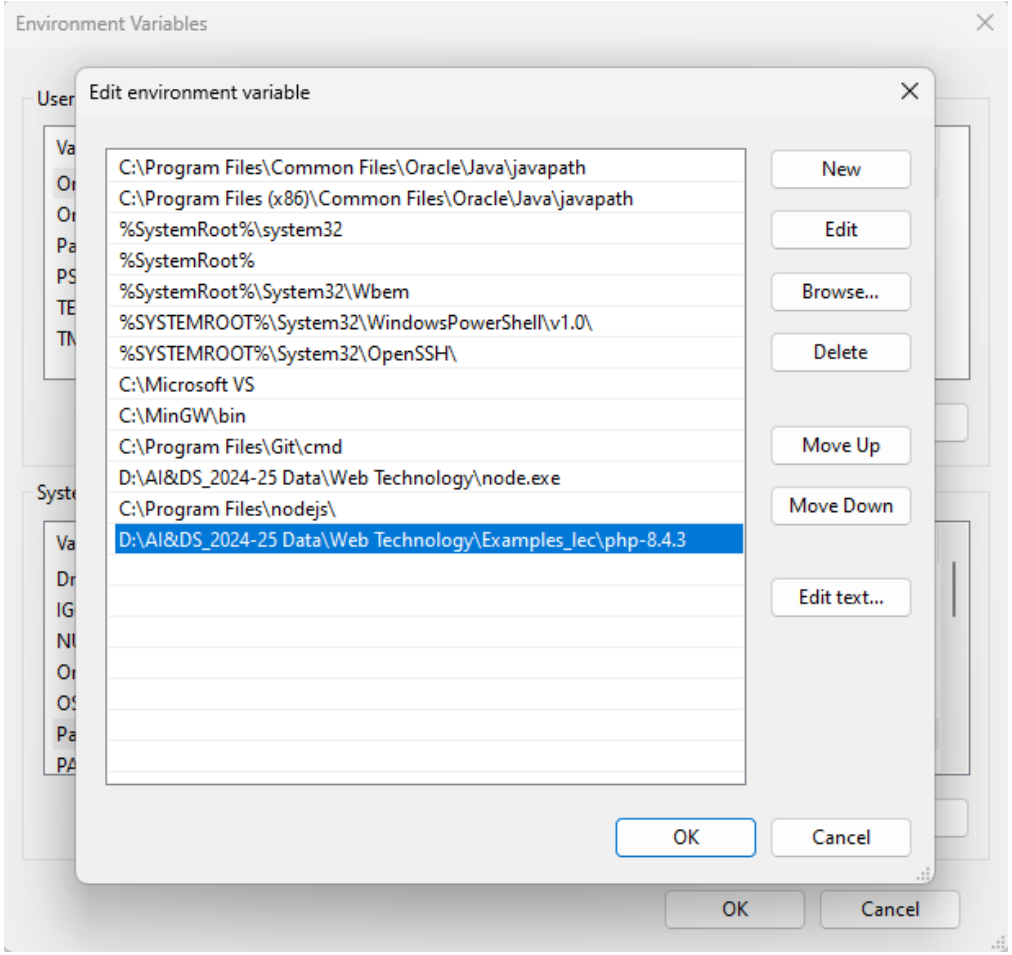
VS17 x64 Thread Safe (2025-Jan-15 11:09:44)

- Zip [32.34MB]
  sha256: 0a46f9ce5535fca15914220d00c91e2ff86a213a7f7ee823c0f357aed597d836
- Debug Pack [37.53MB]
  sha256: b47419987b1c17873b7dfb80b5ae97bf5bbfe487d8aff3fec1482703ec4100b4
- Development package (SDK to develop PHP extensions) [1.35MB]
  sha256: ab5430e09028b25fedef73c2f3017aa0e23058876f3aadb668e011cf0ec29e89

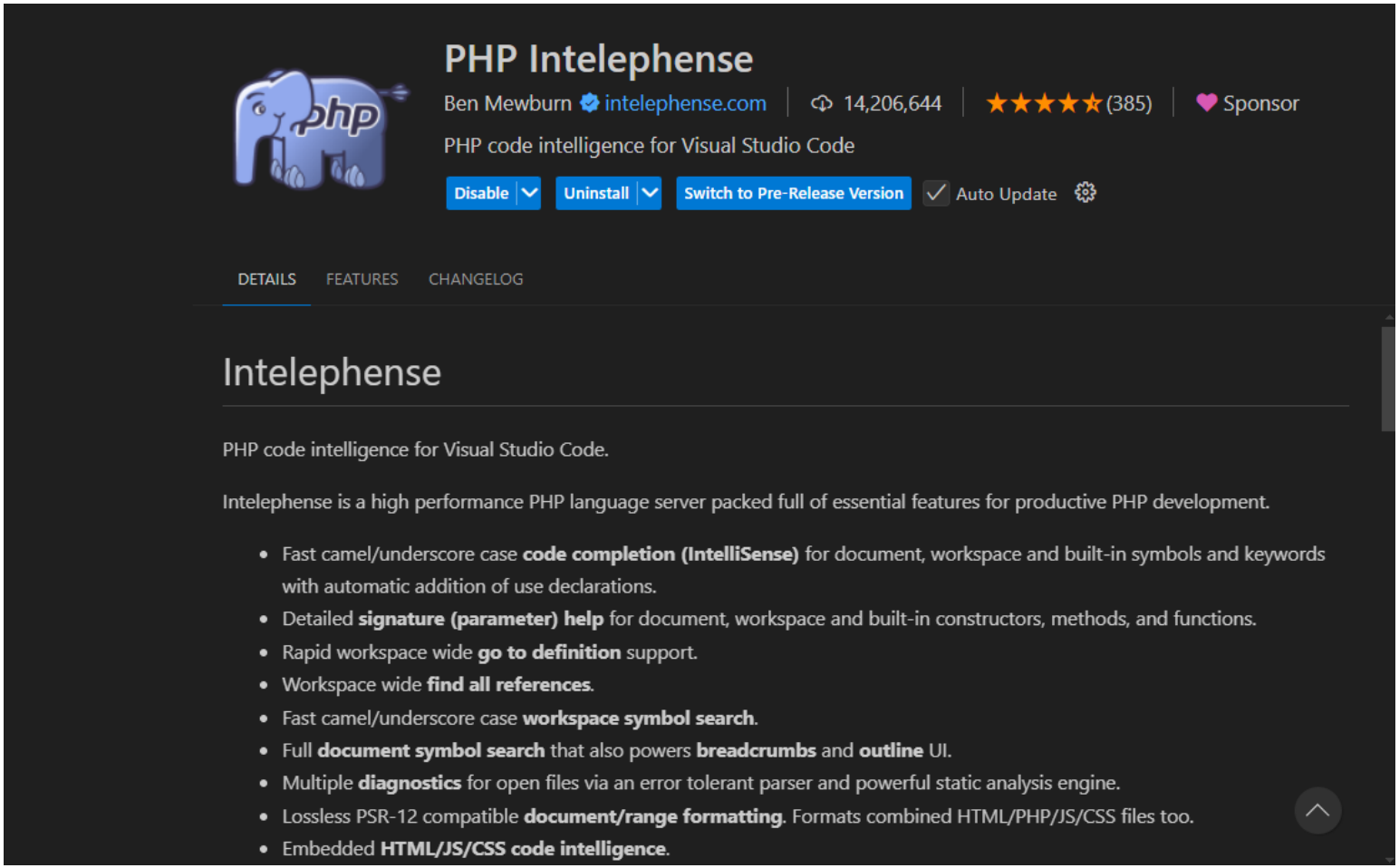
Select **VS17 x64 Thread Safe option to download**

- Select Zip file according to version you want to download.
- Extract the file and rename it with short name.
- Cut and Paste the extracted file to your defined location

Set environment variable



In vs code extensions search for



EXTENSIONS: BUILTIN

@builtin php

PHP Language Basics
Provides syntax highlig...
vscode

PHP Language Featur...
Provides rich language ...
vscode

secphp.php.txt | learn.php | phpfirst.html | trialcode.php | Extension: PHP Intelephense

PHP Intelephense

Ben Mewburn intelephense.com | 14,206,644 | ★★★★☆ (385) | Sponsor

PHP code intelligence for Visual Studio Code

Disable | Uninstall | Switch to Pre-Release Version | Auto Update

DETAILS FEATURES CHANGELOG

Purchase a licence at https://intelephense.com to access PREMIUM features. Licence keys grant a single user access to all current and future premium features on multiple devices *forever*. Licence keys must be activated (automatically) via https before use. Features not marked as PREMIUM are free and always will be free.

The language server client (vscode-intelephense) is open source and licensed under the MIT licence.

The language server (intelephense) is proprietary. Please see here for details.

Quick Start

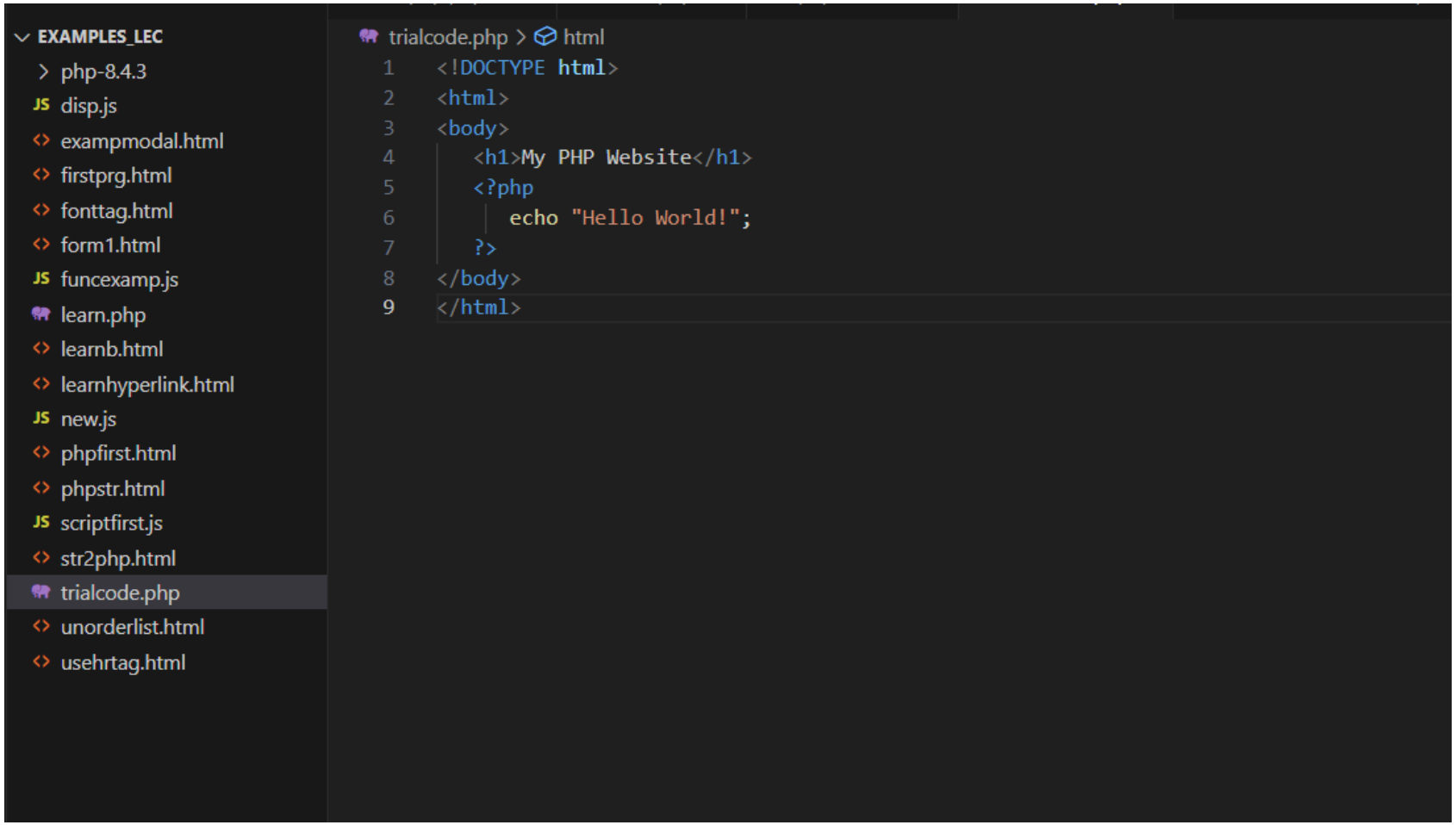
1. Disable the built-in VSCode PHP Language Features.
   - Go to `Extensions`.
   - Search for `@builtin php`
   - Disable `PHP Language Features`. Leave `PHP Language Basics` enabled for syntax highlighting.

   Note that other (3rd party) PHP extensions which provide similar functionality should also be disabled for best results.

2. Add glob patterns for non standard php file extensions to the `files.associations` setting.

- Select PhP language Feature from left hand side and disable it.
- Again in extensions serach for PHP Intelephense and install it.
- Install Code runner through extensions

Write a program and save it as progname.php



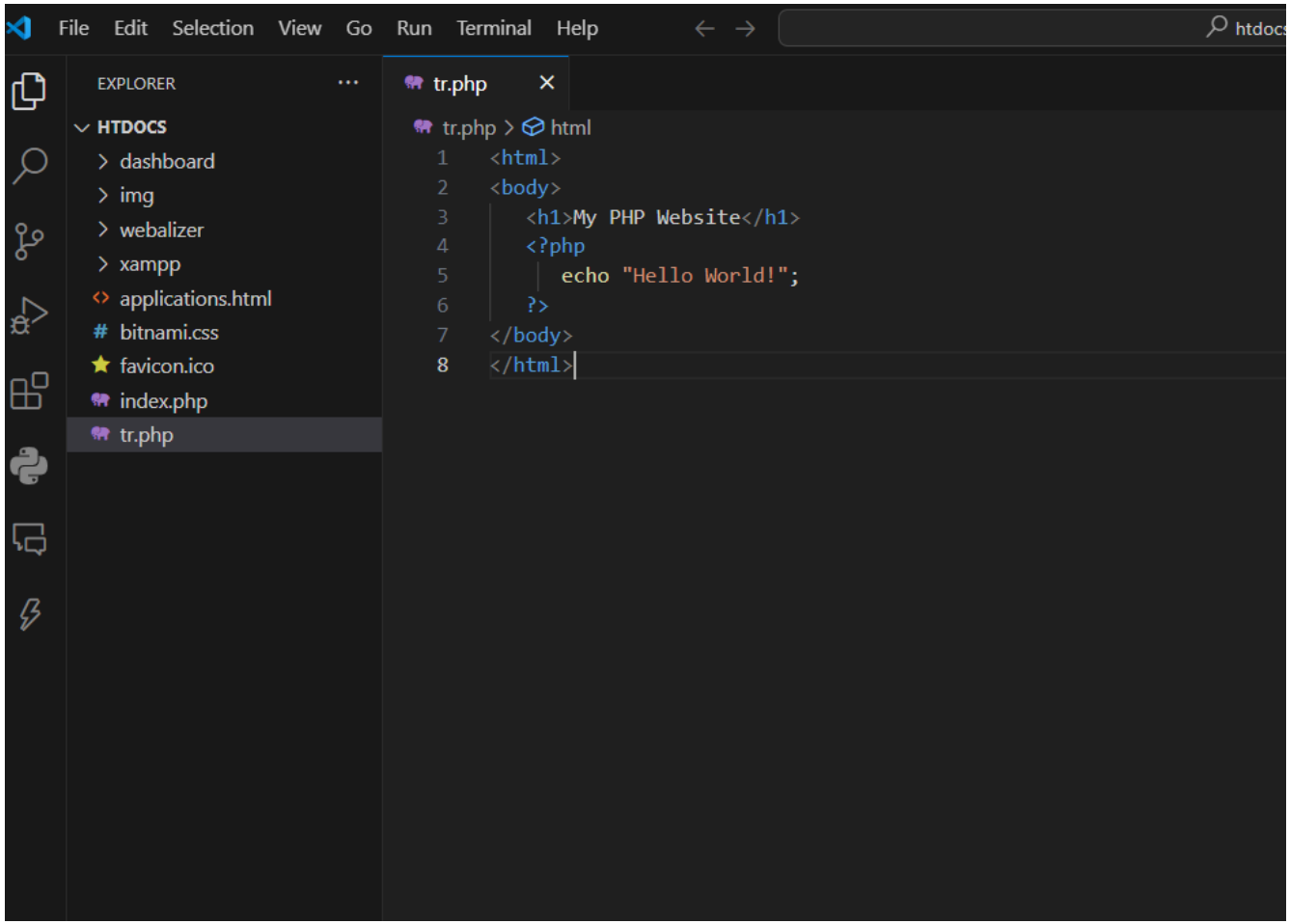
- Press Run button and check the output in terminal.
- Output of PHP code is seen.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\AI&DS_2024-25 Data\Web Technology\Examples_lec> php "d:\AI&DS_2024-25 Data\Web Technology\Examples_lec\trialcode.php"

<!DOCTYPE html>
<html>
<body>
   <h1>My PHP Website</h1>
   Hello World!</body>
</html>
PS D:\AI&DS_2024-25 Data\Web Technology\Examples_lec>
```

File select option open folder and browse for C:\xampp\htdocs



Open browser and in url section provide localhost/filename.php

Creating (Declaring) PHP Variables

- In PHP, a variable starts with the $ sign, followed by the name of the variable.
- Example

```
$x = 5;
$y = "John";
```

PHP Variables

Rules for PHP variables:

- A variable starts with the $ sign, followed by the name of the variable
- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive ($age and $AGE are two different variables)

Output Variables

- The PHP echo statement is often used to output data to the screen.

  Example

  ```
  $x = 5;
  $y = 4;
  echo $x + $y;
  ```

PHP Data Types

- PHP supports the following data types:

1. String
2. Integer
3. Float (floating point numbers - also called double)
4. Boolean
5. Array
6. Object
7. NULL
8. Resource

Double quoted Strings

Double quoted strings perform action on special characters.
E.g. when there is a variable in the string, it returns the *value* of the variable

```
<!DOCTYPE html>
<html>
<body>

<?php
$x = "John";
echo "Hello $x";
?>

</body>
</html>
```

Hello John

Single quoted Strings

Single quoted string literals returns the string as it is

```
<!DOCTYPE html>
<html>
<body>

<?php
$x = "John";
echo 'Hello $x';
?>

</body>
</html>
```

Hello $x

String Length

- The PHP strlen() function returns the length of a string.

Example: echo strlen("Hello world!");

Word Count

- The PHP str_word_count() function counts the number of words in a string.

Example: echo str_word_count("Hello world!");

Search For Text Within a String

- The PHP strpos() function searches for a specific text within a string.
- If a match is found, the function returns the character position of the first match. If no match is found, it will return FALSE.

Example: echo strpos("Hello world!", "world");

How to read user or console input in PHP ?

Method 1: Using **readline() function**

- It is a built-in function in PHP.
- This function is used to read console input.
- **Accept a single input by prompting the user.**
- By default, the data type of the variable accepted through readline() function is **string**.
- So for any other data type, we have to **typecast** it explicitly.

Using **readline() function**

```php
<?php

// For input
// Hello World
$a = readline('Enter a string: ');

// For output
echo $a;
?>
```

Using **readline() function with typecasting**

```php
<?php

// Input section
// $a = 10
$a = (int)readline('Enter an integer: ');

// $b = 9.78
$b = (float)readline('Enter a floating'
            . ' point number: ');

// Entered integer is 10 and
// entered float is 9.78
echo "Entered integer is " . $a
    . " and entered float is " . $b;
?>
```

Accept multiple space separated inputs

- Use another function **explode()** together with **readline()**.
- The first argument of **explode()** is the delimiter we want to use.

Syntax

- explode(*separator,string,limit*)

explode () parameter values

Parameter Values

Parameter	Description
separator	Required. Specifies where to break the string
string	Required. The string to split
limit	Optional. Specifies the number of array elements to return. Possible values: • Greater than 0 - Returns an array with a maximum of *limit* element(s) • Less than 0 - Returns an array except for the last -*limit* elements() • 0 - Returns an array with one element

Example

```
<!DOCTYPE html>
<html>
<body>

<?php
$str = 'one,two,three,four';

// zero limit
print_r(explode(',',$str,0));
print "<br>";

// positive limit
print_r(explode(',',$str,2));
print "<br>";

// negative limit
print_r(explode(',',$str,-1));
?>

</body>
</html>
```

Array ([0] => one,two,three,four)

Array ([0] => one [1] => two,three,four)

Array ([0] => one [1] => two [2] => three)

Method 2: Using fscanf()

- This function works same as the fscanf() function in C.
- We can read 2 integers from Keyboard(STDIN).
- No need to use explicit typecasting for **fscanf() function**, because it is done by the format specifiers , e.g. **%d**, **%f**, **%c** etc.
- fscanf() function is much faster than the readline() function.

Example

```php
<?php

// Input 1 5
fscanf(STDIN, "%d %d", $a, $b);

// Output
// The sum of 1 and 5 is 6
echo "The sum of " . $a . " and "
    . $b . " is " . ($a + $b);
?>
```

Addition of two numbers

```html
<!DOCTYPE html>
<html>
<head>
    <title>Addition of Two Numbers</title>
</head>
<body>

<h2>Addition of Two Numbers</h2>
<form method="post" action="">
    <label for="num1">Number 1:</label>
    <input type="number" id="num1" name="num1" required>
    <br><br>
    <label for="num2">Number 2:</label>
    <input type="number" id="num2" name="num2" required>
    <br><br>
    <input type="submit" name="submit" value="Add">
</form>

<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $num1 = $_POST['num1'];
    $num2 = $_POST['num2'];
    $sum = $num1 + $num2;
    echo "<h3>Result: $num1 + $num2 = $sum</h3>";
}
?>

</body>
</html>
```

PHP Conditional Statements

- In PHP we have the following conditional statements:

1. if statement - executes some code if one condition is true
2. if...else statement - executes some code if a condition is true and another code if that condition is false
3. if...elseif...else statement - executes different codes for more than two conditions
4. switch statement - selects one of many blocks of code to be executed

Practice Programs

1. Write program to accept marks of 5 subjects, calculate its total and percentage. Check student percentage and display its division.
2. Write a PHP program to check whether a person is eligible to vote.

Short Hand If...Else

```php
$a = 13;

$b = $a < 10 ? "Hello" : "Good Bye";

echo $b;
```

PHP Loops

- In PHP, we have the following loop types:

1. while - loops through a block of code as long as the specified condition is true
2. do...while - loops through a block of code once, and then repeats the loop as long as the specified condition is true
3. for - loops through a block of code a specified number of times
4. foreach - loops through a block of code for each element in an array

Practice Programs

1. Write a program to display numbers from 5 to 15 using PHP loop.
2. Write a program to accept a number from user and calculate its factorial and display it.

Example of foreach

```
<!DOCTYPE html>
<html>
<body>

<?php
$colors = array("red", "green", "blue", "yellow");

foreach ($colors as $x) {
  echo "$x <br>";
}
?>

</body>
</html>
```

red
green
blue
yellow

key and value in the foreach definition

```
<!DOCTYPE html>
<html>
<body>

<?php
$members = array("Peter"=>"35", "Ben"=>"37", "Joe"=>"43");

foreach ($members as $x => $y) {
  echo "$x : $y <br>";
}
?>

</body>
</html>
```

Peter : 35

Ben : 37

Joe : 43

Functions

- PHP supports over 1000 built-in functions that can be directly from within a script, to perform a specific task.
- It is also possible to create your own functions. (User defined functions)

Create a function

- A user-defined function declaration starts with keyword function, followed by the name of the function.

Example: function mymsg()

```
{
echo “Hello World”;
}
```

Call a function

- To call a function, write its name followed by parentheses().

Example: function mymsg()

```
{
echo “Hello World”;
}
 mymsg();
```

PHP Function Arguments

- Arguments are specified after the function name, inside the parentheses.
- One can add many arguments as you want, just separate them with a comma.

```html
<!DOCTYPE html>
<html>
<body>

<?php
function sum($x, $y) {
  $z = $x + $y;
  return $z;
}

echo "5 + 10 = " . sum(5,10) . "<br>";
echo "7 + 13 = " . sum(7,13) . "<br>";
echo "2 + 4 = " . sum(2,4);
?>

</body>
</html>
```

5 + 10 = 15
7 + 13 = 20
2 + 4 = 6

Passing Arguments by Reference

- In PHP, arguments are usually passed by value, which means that a copy of the value is used in the function and the variable that was passed into the function cannot be changed.
- When a function argument is passed by reference, changes to the argument also change the variable that was passed in. To turn a function argument into a reference, the & operator is used.

Example of Passing Arguments by Reference

```
<!DOCTYPE html>
<html>
<body>

<?php
function add_five(&$value) {
  $value += 5;
}

$num = 2;
add_five($num);
echo $num;
?>

</body>
</html>
```

```
7
```

Variable Number of Arguments

- By using the ... operator in front of the function parameter, the function accepts an unknown number of arguments.
- This is also called a variadic function.
- The variadic function argument becomes an array.

Example of variadic function.

```
<!DOCTYPE html>
<html>
<body>

<?php
function sumMyNumbers(...$x) {
  $n = 0;
  $len = count($x);
  for($i = 0; $i < $len; $i++) {
    $n += $x[$i];
  }
  return $n;
}

$a = sumMyNumbers(5, 2, 6, 2, 7, 7);
echo $a;
?>

</body>
</html>
```

29

Arrays in PHP

Introduction

- An array in PHP is an ordered map that associates values to keys.
- A PHP array can be used to implement different data structures such as a stack, queue, list (vector), hash table, dictionary, etc.
- The value part of an array element can be other arrays. This fact can be used to implement tree data structure and multidimensional arrays.

Built-in Array

- The built-in array() function uses the parameters given to it and returns an object of array type.
- One or more comma-separated parameters are the elements in the array.

Syntax: array(mixed ...$values): array

- Each value in the parenthesis may be either a singular value (it may be a number, string, any object or even another array), or a key-value pair.
- The association between the key and its value is denoted by the "=>" symbol.

Example

```php
$arr1 = array(10, "asd", 1.55, true);
$arr2 = array("one"=>1, "two"=>2, "three"=>3);
$arr3 = array(
   array(10, 20, 30),
   array("Ten", "Twenty", "Thirty"),
   array("physics"=>70, "chemistry"=>80, "maths"=>90)
);
```

Using Square Brackets [ ]

- Instead of the array() function, the comma-separated array elements may also be put inside the square brackets to declare an array

```
$arr1 = [10, "asd", 1.55, true];
$arr2 = ["one"=>1, "two"=>2, "three"=>3];
$arr3 = [ [10, 20, 30],
   ["Ten", "Twenty", "Thirty"],
   ["physics"=>70, "chemistry"=>80, "maths"=>90] ];
```

Types of Arrays in PHP

- **Indexed Array** – An array which is a collection of values only is called an indexed array. Each value is identified by a positional index staring from "0". Values are stored and accessed in linear fashion.
- **Associative Array** – If the array is a collection of key-value pairs, it is called as an associative array. The key component of the pair can be a number or a string, whereas the value part can be of any type. Associative arrays store the element values in association with key values rather than in a strict linear index order.
- **Multi Dimensional Array** – If each value in either an indexed array or an associative array is an array itself, it is called a multi dimensional array. Values are accessed using multiple indices

Accessing the Array Elements

- To access any element from a given array, you can use the array[key] syntax.

```php
<?php
   $arr1 = [10, 20, 30];
   $arr2 = array("one"=>1, "two"=>2, "three"=>3);

   var_dump($arr1[1]);
   var_dump($arr2["two"]);
?>
```

:t will produce the following **output** –

```
int(20)
int(2)
```

Traversing an Indexed Array in PHP

- Any type of PHP loop can be employed to traverse an array.
- If we want to use a **for** or **while** loop, we have to find the number of elements in the array with count() function and use its value as the test condition for the counted **for** or **while** loop.

The following code uses a **for** loop to list all the elements in an indexed array.

```php
<?php
   $numbers = array(10, 20, 30, 40, 50);

   for ($i=0; $i<count($numbers); $i++){
      echo "numbers[$i] = $numbers[$i] \n";
   }
?>
```

PHP Files and I/O

PHP - Files & I/O

- Functions related to files are:

1. Opening a File
2. Reading a File
3. Writing a File
4. Closing a File

Opening a File

- The PHP **fopen()** function is used to open a file.
- It requires two arguments stating first the file name and then mode in which to operate.

Read File - fread()

- The fread() function reads from an open file.
- The first parameter of fread() contains the name of the file to read from and the second parameter specifies the maximum number of bytes to read.

Close File - fclose()

- The fclose() function is used to close an open file.
- The fclose() requires the name of the file (or a variable that holds the filename) we want to close.

Files modes can be specified as one of the six options

Modes	Description
r	**Open a file for read only**. File pointer starts at the beginning of the file
w	**Open a file for write only**. Erases the contents of the file or creates a new file if it doesn't exist. File pointer starts at the beginning of the file
a	**Open a file for write only**. The existing data in file is preserved. File pointer starts at the end of the file. Creates a new file if the file doesn't exist
x	**Creates a new file for write only**. Returns FALSE and an error if file already exists
r+	**Open a file for read/write**. File pointer starts at the beginning of the file
w+	**Open a file for read/write**. Erases the contents of the file or creates a new file if it doesn't exist. File pointer starts at the beginning of the file
a+	**Open a file for read/write**. The existing data in file is preserved. File pointer starts at the end of the file. Creates a new file if the file doesn't exist
x+	**Creates a new file for read/write**. Returns FALSE and an error if file already exists

Example to read a file

```
<!DOCTYPE html>
<html>
<body>

<?php
$myfile = fopen("webdictionary.txt", "r") or die("Unable to open file!");
echo fread($myfile,filesize("webdictionary.txt"));
fclose($myfile);
?>

</body>
</html>
```

AJAX = Asynchronous JavaScript and XML CSS = Cascading Style Sheets HTML = Hyper Text Markup Language PHP = PHP Hypertext Preprocessor SQL = Structured Query Language SVG = Scalable Vector Graphics XML = EXtensible Markup Language

Read Single Line - fgets()

- The fgets() function is used to read a single line from a file.

```
<!DOCTYPE html>
<html>
<body>

<?php
$myfile = fopen("webdictionary.txt", "r") or die("Unable to open file!");
echo fgets($myfile);
fclose($myfile);
?>

</body>
</html>
```

AJAX = Asynchronous JavaScript and XML

Check End-Of-File - feof()

- The feof() function checks if the "end-of-file" (EOF) has been reached.
- The feof() function is useful for looping through data of unknown length.

Example of feof()

```
<!DOCTYPE html>
<html>
<body>

<?php
$myfile = fopen("webdictionary.txt", "r") or die("Unable to open file!");
// Output one line until end-of-file
while(!feof($myfile)) {
  echo fgets($myfile) . "<br>";
}
fclose($myfile);
?>

</body>
</html>
```

AJAX = Asynchronous JavaScript and XML
CSS = Cascading Style Sheets
HTML = Hyper Text Markup Language
PHP = PHP Hypertext Preprocessor
SQL = Structured Query Language
SVG = Scalable Vector Graphics
XML = EXtensible Markup Language

Read Single Character - fgetc()

- The fgetc() function is used to read a single character from a file.

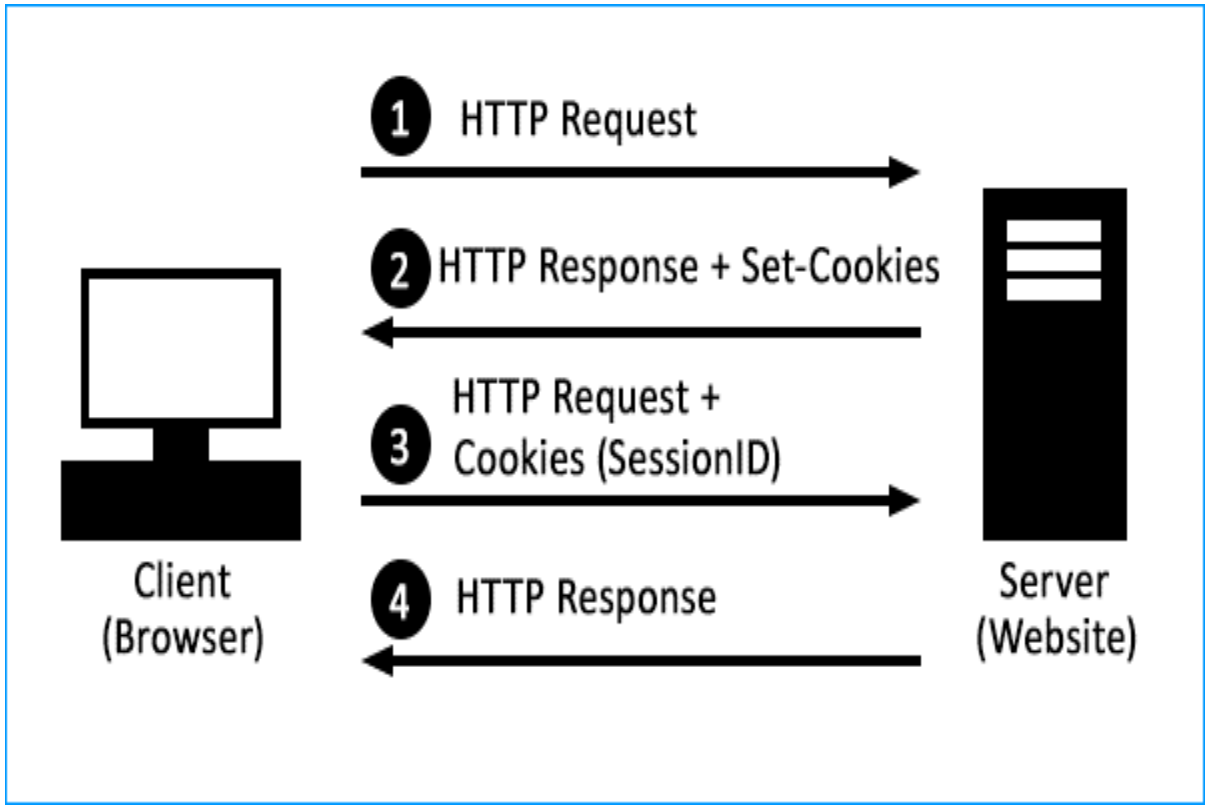
```php
<?php
$myfile = fopen("webdictionary.txt", "r") or die("Unable to open file!");
// Output one character until end-of-file
while(!feof($myfile)) {
  echo fgetc($myfile);
}
fclose($myfile);
?>
```

Write to File - fwrite()

- The fwrite() function is used to write to a file.
- The first parameter of fwrite() contains the name of the file to write to and the second

```php
<?php
$myfile = fopen("newfile.txt", "w") or die("Unable to open file!");
$txt = "John Doe\n";
fwrite($myfile, $txt);
$txt = "Jane Doe\n";
fwrite($myfile, $txt);
fclose($myfile);
?>
```

Understanding Cookies



Cookies

- A cookie is often used to identify a user.
- A cookie is a small file that the server embeds on the user's computer.
- Each time the same computer requests a page with a browser, it will send the cookie too.
- With PHP, you can both create and retrieve cookie values.

Create Cookies

- A cookie is created with the setcookie() function.

Syntax

```
setcookie(name, value, expire, path, domain, secure, httponly);
```

- Only the name parameter is required. All other parameters are optional.

Create/Retrieve a Cookie

- The following example creates a cookie named "user" with the value "John Doe".
- The cookie will expire after 30 days (86400 * 30). The "/" means that the cookie is available in entire website (otherwise, select the directory you prefer).
- We then retrieve the value of the cookie "user" (using the global variable $_COOKIE). We also use the isset() function to find out if the cookie is set

Example

```php
<?php
$cookie_name = "user";
$cookie_value = "John Doe";
setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/"); // 86400 = 1 day
?>
<html>
<body>

<?php
if(!isset($_COOKIE[$cookie_name])) {
  echo "Cookie named '" . $cookie_name . "' is not set!";
} else {
  echo "Cookie '" . $cookie_name . "' is set!<br>";
  echo "Value is: " . $_COOKIE[$cookie_name];
}
?>

</body>
</html>
```

What are cookies used for?

- **User sessions:** Cookies help associate website activity with a specific user. A session cookie contains a unique string (a combination of letters and numbers) that matches a user session with relevant data and content for that user.
- **Personalization:** Cookies help a website "remember" user actions or user preferences, enabling the website to customize the user's experience.

What are cookies used for?

- **Tracking:** Some cookies record what websites users visit. This information is sent to the server that originated the cookie the next time the browser has to load content from that server. With third-party tracking cookies, this process takes place anytime the browser loads a website that uses that tracking service.

Delete a Cookie

- To delete a cookie, use the setcookie() function with an expiration date in the past

```
<?php
// set the expiration date to one hour ago
setcookie("user", "", time() - 3600);
?>
<html>
<body>

<?php
echo "Cookie 'user' is deleted.";
?>

</body>
</html>
```

Session

- “Session” is the term used to refer to a visitor’s time browsing a web site. It’s meant to represent the time between a visitor’s first arrival at a page on the site and the time they stop using the site.

Session Management

- Client sends multiple requests to server
- Save some “state” information
  - logged in
  - choice of background colour
  - ...
- Server customizes responses based on client session information

Storage:

- Client-side session: completely stored in cookie
- Server-side session: stored on server, looked up from cookie

Sessions

- A session is a way to store information (in variables) to be used across multiple pages.
- Unlike a cookie, the information is not stored on the users computer.
- Session variables store user information to be used across multiple pages (e.g. username, favorite color, etc).
- By default, session variables last until the user closes the browser.
- So; Session variables hold information about one single user, and are available to all pages in one application.

PHP Session

- A session is started with the session_start() function.
- Session variables are set with the PHP global variable: $_SESSION.
- The session_start() function must be the very first thing in your document. Before any HTML tags.

Example to start a Session

```php
<?php
// Start the session
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// Set session variables
$_SESSION["favcolor"] = "green";
$_SESSION["favanimal"] = "cat";
echo "Session variables are set.";
?>

</body>
</html>
```

Session variables are set.

Get PHP Session Variable Values

- session variables are not passed individually to each new page, instead they are retrieved from the session we open at the beginning of each page (session_start()).
- All session variable values are stored in the global $_SESSION variable

Example

```php
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// Echo session variables that were set on previous page
echo "Favorite color is " . $_SESSION["favcolor"] . ".<br>";
echo "Favorite animal is " . $_SESSION["favanimal"] . ".";
?>

</body>
</html>
```

Favorite color is green.
Favorite animal is cat.

Destroy a PHP Session

- To remove all global session variables and destroy the session,
  use session_unset() and session_destroy()

Example

```php
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// remove all session variables
session_unset();

// destroy the session
session_destroy();

echo "All session variables are now removed, and the session is destroyed."
?>

</body>
</html>
```

All session variables are now removed, and the session is destroyed.